MSDM 5001
An Introduction to Computational and Modelling Tools

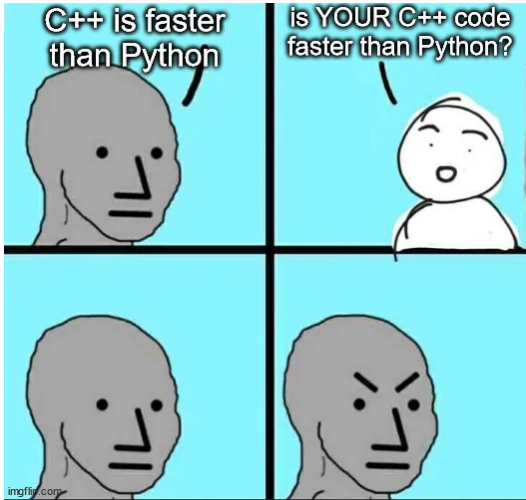
Lecture 8: Introduction to numpy, numba and Cython, Part I

Installation

- Install relevant packages:
  - `uv pip install numpy numba cython setuptools jupyter ipykernel`
  - To use Cython, you need to make sure you have a C/C++ compiler properly installed on your system

`numpy` – Python's numerical array library

- Fundamental data structure: numpy arrays
  - Contiguous in memory (compliant with Python's buffer protocol)
  - Row-major (default, can be changed) ← why does this matter?
  - Uniform data type
  - Size fixed at creation
  - Multi-dimensional
  - Supports vectorized operations
    (i.e. no explicit Python loops + potential to use SIMD)
- Fast: basically a Python wrapper for efficient C codes and libraries



```
import numpy as np
```

Creating arrays in numpy

- From existing data: `np.array([1, 2, 3])`
- Filled arrays:
  - `np.zeros((3, 4))`
  - `np.ones((3, 4))`
  - `np.empty((3, 4))`
- Sequences/ranges:
  - `np.arange(start, stop, step)`
  - `np.linspace(start, stop, num)`

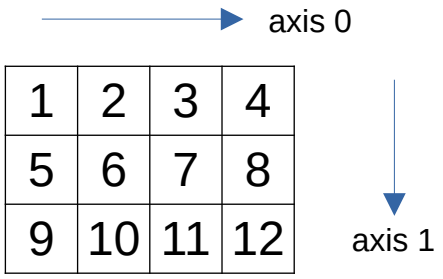
Indexing and slicing numpy arrays

- Basic indexing
  - `arr[0], arr[-1], arr[2, 2]`
- Slicing
  - `arr[1:-1], arr[2:], arr[:, 2], arr[:, 2:4]`
  - Creates “views” of the original array, any changes will be reflected
- Fancy indexing and boolean indexing
  - `arr[[2, 5, -1]]`
  - `arr[arr > 3]`

Axes of numpy arrays

- Consider a 2D array

```
[[1, 2,  3,  4],
 [5, 6,  7,  8],
 [9, 10, 11, 12]]
```



- Can be generalized to higher dimensions

```
[[[1, 2,  3,  4],
  [5, 6,  7,  8],
  [9, 10, 11, 12]],
 [[2,  4,   6,  8],
  [10, 12,  14, 16],
  [18, 20,  22, 24]]]
```

shape: (2, 3, 4)

- `arr[0, :, :] = ?`
- `arr[:, 0, :] = ?`
- `arr[:, :, 0] = ?`
- `arr[0, 1, 1] = ?`

Various flags of a numpy array

- C_CONTIGUOUS: True if using C-ordering (default)
- F_CONTIGUOUS: True if using Fortran-ordering
- OWNDATA : True if array owns its memory buffer
  - Usually true for normally created arrays
  - False for views (see later)
- WRITEABLE: True if you can modify array values

Exercise 1

1. Create an array representing test scores for 30 students (random integers 0-100):

   ```
   scores = np.random.randint(low=0, high=101, size=30)
   ```

2. Find all scores above 80 and increase them by 5 bonus points, but cap at 100
3. Find all failing scores (below 60) and replace with 60
4. Find all students whose scores are between 70 and 90, print out their indices in `scores`

Basic operations on numpy arrays

- Element-wise operations:
  - `a+b`, `a-b`, `a*b`, `a/b`, `a**2`
- Basic aggregation:
  - `np.sum(a)`, `np.min(a)`, `np.max(a)`, `np.mean(a)`, `np.std(a)`
- Universal functions (ufuncs)
  - Functions that operate on arrays element-wise
  - e.g. `np.sin(a)`, `np.log(a)`, `np.exp(a)`, `np.abs()`
  - Can define custom ufuncs with np.vectorize, but less prominent performance benefits

Basic operations on numpy arrays

- Aggregation/reduction operations accept `axis` keyword:
  - e.g. `np.sum(a, axis=1)`
    This only sums along axis 1

Broadcasting

- What happens when we add a number to an array?

```
arr = np.array([1, 2, 3])
arr + 1
```

- Supposed to get error due to incompatible shapes, but…
- numpy automatically “broadcasts” the number 1 into a compatible shape:

```
1 -> [1, 1, 1]
```

Axes of numpy arrays

- Can add new axes to an array:

```
a = np.array([1, 2, 3])
b = a[:, np.newaxis]
b = a[:, None]    # equivalent
```

- Result: add a new dimension of size 1
  - In the example above, `b` will have shape (3, 1)

Broadcasting

- Example
  - `arr1 = [[1, 2, 3], [4, 5, 6]]`, `arr2 = [10, 20, 30]`
  - `arr1` has shape (2, 3), and `arr2` has shape (3)
  - When we add the two, `arr2` will be “stretched” to have shape (2, 3):
    `arr2 -> [[10, 20, 30], [10, 20, 30]]`
  - Final result: `[[11, 22, 33], [14, 25, 36]]`

Broadcasting

- A more complex example
  - What will `result` be?

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30, 40])

a_new = a[:, np.newaxis]
result = a_new + b
```

Shape manipulation

- `np.reshape(arr, shape, order)`
  - shape: int or tuple of ints
  - order: ‘C’ for C-order, ‘F’ for Fortran-order
  - Preserves “order” of elements (i.e. underlying memory layout remains the same)
- `np.transpose(arr)`
  - Swaps rows and columns
  - Returns a view, does NOT change underlying memory layout

Shape manipulation

- `np.flatten(arr)` vs `np.ravel(arr)`
  - `np.flatten` always returns a copy
  - `np.ravel` returns a view if possible, more efficient
- When does `np.ravel` return a view?
  - If the array has a matching memory order (default C-ordering)
- When does `np.ravel` return a copy?
  - If the array is non-contiguous (e.g. after transpose, slicing etc.)

Broadcasting

- numpy compares shapes element-by-element from right to left
- Rules
  - Two dimensions are compatible when either (i) they are equal, or (ii) one of them is 1
  - Non-existent dimension is treated as 1 (e.g. in `arr + 1`)
  - Dimension of size 1 is “stretched” to match dimension of the other
  - Incompatible dimensions -> error

Exercise 2

- Given
```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
```

1. What is `arr[[0, 2], [0, 2]]`? Does it return a copy or a view?
2. What is `arr[[0, 2]][:, [0, 2]]`? Does it return a copy or a view?
3. Does np.ravel(arr.transpose()) return a copy or a view?

Exercise 3

- You are given an n x d array:
  - n = number of students, d = number of courses
  - Each entry: score of a particular student in a particular course

1. Find the standard deviation for different courses. Return an array of shape (d,)
2. Find the average score across classes for each student. Return an array of shape (1, n)
3. For each student in each course, compute their z-score. Return an array of shape (n, d)

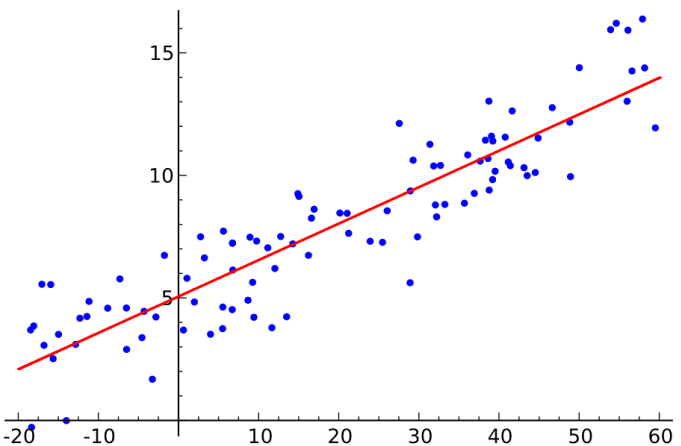
Linear algebra with numpy

- Matrix multiplication
  - `A@B` or `np.dot(A, B)`
- Solving linear systems
  - `np.linalg.solve(A, b)`
- Solving eigen problems
  - `np.linalg.eig(A)`,`np.linalg.svd(A)`

Exercise 4: Least square

- Given an n x d array A and an n-dimensional vector x
  - Find a d-dimensional vector β such that $\|A\beta - x\|^2$ is minimized

1. Find a closed-form expression for A
2. You are given A and x stored in csv files. Load them as numpy arrays using `np.loadtxt()`.
3. Using numpy's solver, find β



numba – A JIT compiler for Python

- Sometimes Python loops are unavoidable…

```
def func(n):
    if n == 1 or n == 2:
        return 1

    a, b = 1, 1
    for i in range(2, n):
        a, b = b, a+b
    return b
```

- Basically impossible to “vectorize” with numpy :(

numba – A JIT compiler for Python

- The “main” reason why Python is slow…

```
def add_numbers(a, b):
    return a + b
```

- In C/C++ or other compiled languages, this is basically just one CPU operations
- What a Python interpreter does every time this function is called:
  - Checks types of a and b
  - Find the right + operation for those types
  - Performs addition
  - Repackage the result into a Python integer object

Dilemma of Python:
Easy to write, but slow to run

numba – A JIT compiler for Python

- JIT compilation – “Just-in-time” compilation
  - Compile Python code to machine code just before it runs for the first time
  - Subsequent calls are much faster
- Very easy to use, just a decorator! See demo.

numba – A JIT compiler for Python

- “nopython” mode – fastest compilation mode
  - Does not involve any Python’s C API
  - Directly compiles to machine code
  - More efficient memory management
  - Enables compiler optimizations
  - More restrictive, some python features not supported
- Should always be used as your default!
  - `@numba.njit`

What numba likes

- Core numerical computing
  - numpy arrays and associated operations
  - Numeric types: int, float, bool etc.
  - Math functions: math.sqrt, math.sin etc.
- Basic Python
  - Control flow: if, else, while, break continue
  - Basic arithmetic operations
  - Tuples

What numba doesn’t like

- Dynamic python features
  - Regular list, dict, set
  - Strings and string operations
  - print()
  - Classes
- Standard library
  - os, sys, json, pickle
- Third party libraries

Example: Conway’s Game of life

- Consider an m by n grid of cells.
- Each cell either has value 1 (alive) or value 0 (dead).
- Update rule:
  - Any cell alive with 2-3 alive neighbors survives
  - Any dead cell with exactly 3 neighbors becomes alive
  - All other cells die or stay dead
- Unclear how to use numpy for it. Pure Python seems unavoidable.
- See demo.